

1. 今日の位置づけ（第3回との分担）

第3回では DAO で **SELECT（参照）** を PreparedStatement により安全に扱いました。

第4回では同じく DAO に閉じて **INSERT / UPDATE / DELETE（更新系）** を実行し、複数の更新を **1つのトランザクション** にまとめる方法を扱います。

題材は引き続き lesson DB の **emps**（および既存の **depts**）。Java サンプルは **Unit03_DAO/src** の **DaoBase / EmpDto / EmpDao** をそのまま使います。トランザクションの実行例は **Unit04_DAO_Update/src/TransactionSample.java** を参照してください。

2. イメージで掴む：銀行送金のたとえ

トランザクションは、ひとことで言うと「**全部成功か、全部なかったことに（all-or-nothing）**」を保証する仕組みです。

A口座から B口座へ 1,000 円を送金する場面で考えてみましょう。引き出し（A）と入金（B）の2つの更新のうち、片方だけ成功すると **お金が消える** 事態になります。

A口座 → B口座 へ 1,000円送金（2つの更新で1つの仕事）

× トランザクション無し

1) A口座から 1,000円 引き出し
UPDATE accounts SET balance = balance - 1000 ... ✓ 成功

2) B口座に 1,000円 入金
UPDATE accounts SET balance = balance + 1000 ... ✗ 失敗
(DB切断・想定外エラー など)

結果：1,000円 が宙に消える

A口座：10,000 → 9,000円（1,000減）
B口座：5,000 → 5,000円（変化なし）

→ A側だけ反映の「中途半端」な状態
→ 不整合データが残り、追跡も困難

○ トランザクション有り

1) A口座から 1,000円 引き出し
UPDATE accounts SET balance = balance - 1000 ... ✓ 成功

2) B口座に 1,000円 入金
UPDATE accounts ... ✗ 失敗
→ catch (SQLException) で con.rollback()

結果：両口座とも元のまま

A口座：10,000 → 10,000円（rollbackで戻る）
B口座：5,000 → 5,000円（そもそも未反映）

→ 「全部成功 か 全部なかったことに」
→ all-or-nothing が守られる

この「片方だけ反映されると壊れる」関係になっている複数SQLを、**同じConnection上でひとまとまり**に扱い、最後にまとめて **commit（確定） / rollback（取消）** するのが **トランザクション** です。

3. 更新系の型 (executeUpdate と更新件数)

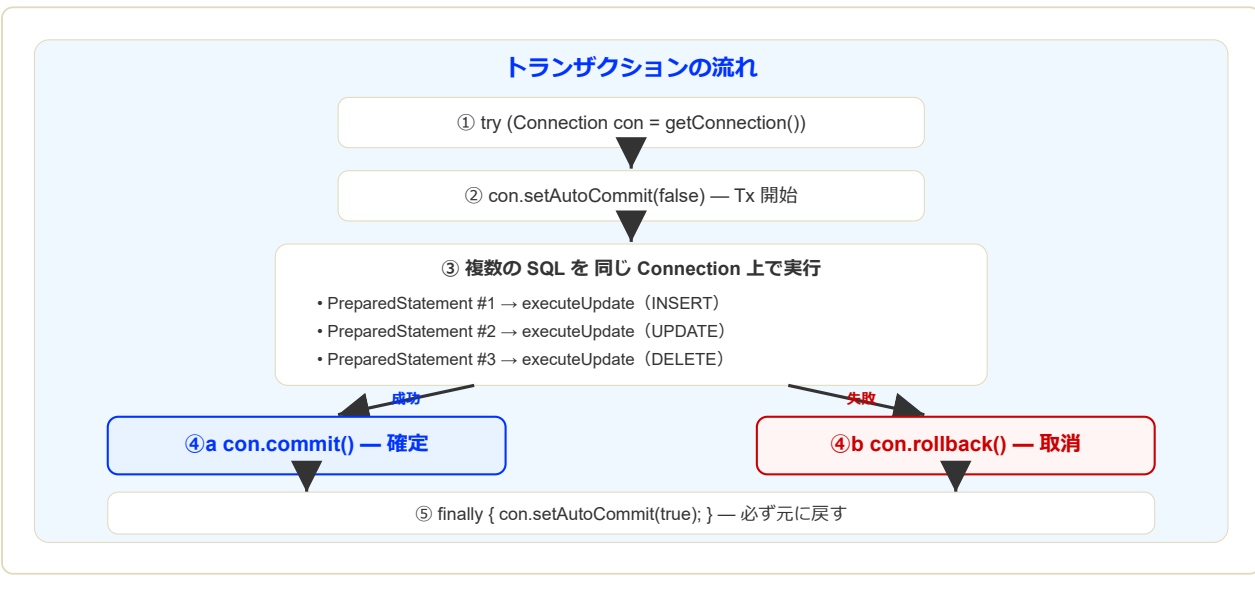
- 参照系は `executeQuery()` → `ResultSet`。
- 更新系は `executeUpdate()` → `int` (影響した行数 = 更新件数)。
- `EmpDao#insert / updateSalaryAndDept / delete` はいずれも `PreparedStatement` で ? に値をバインドする (第3回と同じ原則)。

4. トランザクションの基本 (setAutoCommit / commit / rollback)

JDBC のお約束 (この講義で使う型)

- 開始前に `setAutoCommit(false)` (自動コミットを止める = Tx 開始)。
- 同じ `Connection` に対して複数の `PreparedStatement` を実行する。
- 全部成功なら `con.commit()`、途中で失敗なら `con.rollback()`。
- `finally` で `setAutoCommit(true)` に戻す (後続処理に影響させない)。

図解：トランザクションの流れ

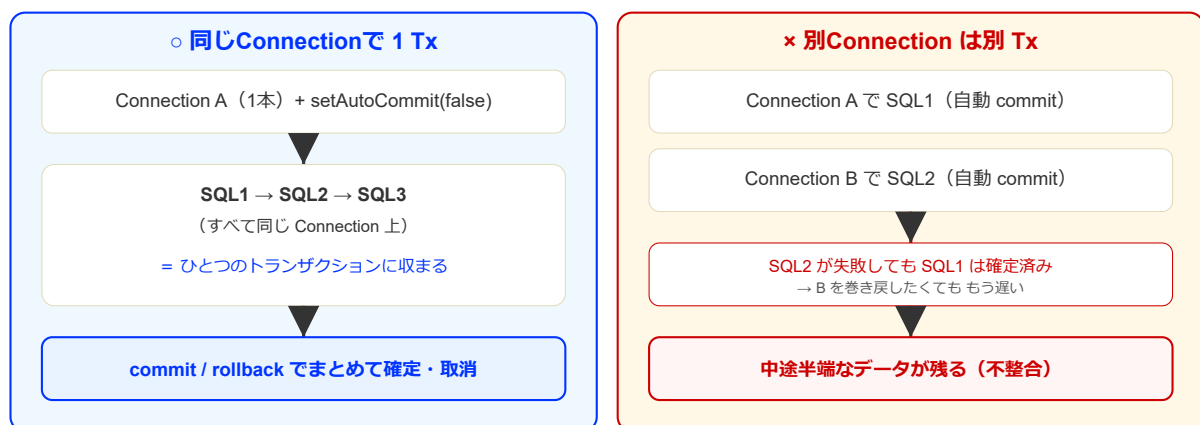


5. 鉄則：同じ Connection でまとめる

トランザクションの単位は「**Connection**」です。別々の Connection を使うと **別々のトランザクション**になり、まとめて取り消すことができません。

補足：EmpDao の各メソッドは内部で `getConnection()` しているため、**そのままではトランザクションに使用できません**。発展課題では **1本の Connection を取り、SQL を直接並べるか、Connection を引数に取る DAO メソッドを自分で追加**します。

図解：同一Connection と 別Connection の比較

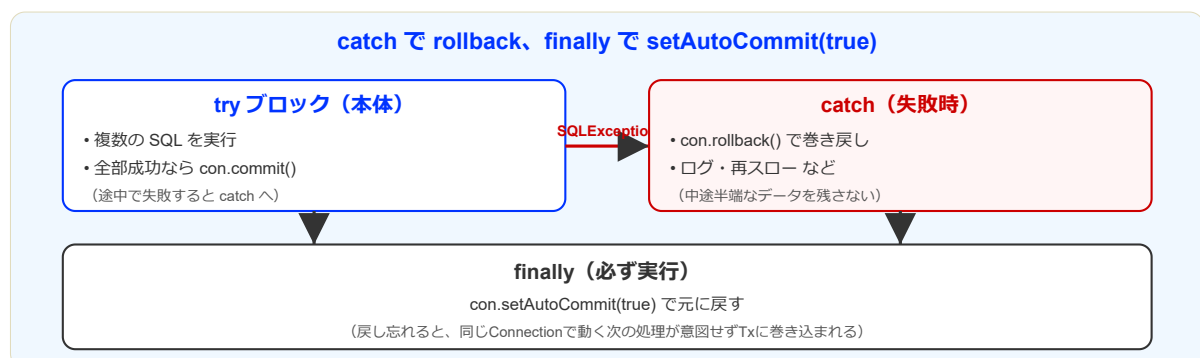


6. 例外処理と rollback の位置

`try` の本体で複数 SQL を順に走らせ、すべて通れば末尾で `commit`。途中で `SQLException` が出たら `catch` で `rollback`。最後に `finally` で `setAutoCommit(true)` に戻すまでがワンセットです。

教材ではまず原因が見えるよう `e.printStackTrace()` を入れますが、実務ではログ設計・通知・リカバリを別途検討します。

図解：try / catch / finally と rollback



7. サンプルコード（段階的に難しく）

狙い まずは「更新系の最小形（executeUpdate）」→次に「同じ Connection を使ったトランザクション」を、2段階で写経します。

▼ (1) 単純な更新系 (UPDATE 1本)

ポイント: `executeUpdate()` の戻り値 (更新件数) を表示する。

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SimpleUpdateSample extends DaoBase {

    public static void main(String[] args) {
        SimpleUpdateSample app = new SimpleUpdateSample();
        app.run();
    }

    private void run() {
        String sql = "UPDATE emps SET salary = ? WHERE emp_id = ?";

        try (Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, 290000);
            ps.setInt(2, 3);

            int n = ps.executeUpdate();
            System.out.println("update件数=" + n);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

マウスで全選択して貼り付け欄へドラッグしてください:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SimpleUpdateSample extends DaoBase {

    public static void main(String[] args) {
        SimpleUpdateSample app = new SimpleUpdateSample();
```

▼ (2) トランザクション込み（同一Connectionで複数操作）

ポイント： `setAutoCommit(false)` → 成功で `commit`、失敗で `rollback`、最後に `true` に戻す。

```

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class TransactionUpdateSample extends DaoBase {

    public static void main(String[] args) {
        TransactionUpdateSample app = new TransactionUpdateSample();
        app.run();
    }

    private void run() {
        int empId = 99;

        String insertSql = "INSERT INTO emps (emp_id, fname, lname, salary, dept_no) VALUES (?, ?, ?, ?, ?)";
        String selectSql = "SELECT emp_id, fname, lname, salary, dept_no FROM emps WHERE emp_id = ?";
        String updateSql = "UPDATE emps SET salary = ? WHERE emp_id = ?";
        String deleteSql = "DELETE FROM emps WHERE emp_id = ?";

        try (Connection con = getConnection()) {
            con.setAutoCommit(false);
            try {
                // 1) INSERT
                try (PreparedStatement ps = con.prepareStatement(insertSql)) {
                    ps.setInt(1, empId);
                    ps.setString(2, "Tx");
                    ps.setString(3, "Sample");
                    ps.setInt(4, 200000);
                    ps.setInt(5, 10);
                    System.out.println("insert件数=" + ps.executeUpdate());
                }

                // 2) SELECT
                selectAndPrint(con, selectSql, empId);

                // 3) UPDATE
                try (PreparedStatement ps = con.prepareStatement(updateSql)) {
                    ps.setInt(1, 210000);
                    ps.setInt(2, empId);
                    System.out.println("update件数=" + ps.executeUpdate());
                }

                // 4) SELECT
                selectAndPrint(con, selectSql, empId);
            }
        }
    }
}

```

```

        // 5) DELETE
        try (PreparedStatement ps = con.prepareStatement(deleteS
            ps.setInt(1, empId);
            System.out.println("delete件数=" + ps.executeUpdate()
        }

        // 6) SELECT (該当なし確認)
        selectAndPrint(con, selectSql, empId);

        con.commit();
    } catch (SQLException e) {
        con.rollback();
        e.printStackTrace();
    } finally {
        con.setAutoCommit(true);
    }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

private void selectAndPrint(Connection con, String sql, int empId) throws SQLExc
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, empId);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                System.out.println("emp_id=" + rs.getInt("emp_id")
                    + " fname=" + rs.getString("fname")
                    + " lname=" + rs.getString("lname")
                    + " salary=" + rs.getInt("salary")
                    + " dept_no=" + rs.getInt("dept_no")
            } else {
                System.out.println("該当なし: emp_id=" + empId);
            }
        }
    }
}
}
}
}
}

```


マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class TransactionUpdateSample extends DaoBase {

    public static void main(String[] args) {
```

8. 穴埋め～トランザクションの骨格

下線部（ ）を埋めてから、解答を開いてください。

```
try (Connection con = getConnection()) {
    con.setAutoCommit( (1) );
    try {
        // 複数の PreparedStatement 実行
        con.(2)();
    } catch (SQLException e) {
        con.(3)();
        // 教材ではまず原因が見えるようにする（本来はログ設計・通知・リカバリー等）
        e.printStackTrace();
    } finally {
        con.setAutoCommit( (4) );
    }
}
```

(1)～(4) に入る語を答えよ（Java のリテラルまたはメソッド名）。

▼ 解答

- (1) false
- (2) commit
- (3) rollback
- (4) true

9. 理解チェック

問題 次の問いに答えよ（口頭でOK）。

- (1) なぜ `finally` で `setAutoCommit(true)` に戻す必要がある？（1行）
- (2) `commit` は「いつ」呼ぶ？ `rollback` は「いつ」呼ぶ？（それぞれ1行）
- (3) なぜ「複数の更新をまとめる」には **同じ Connection** が必要？（1行）
- (4) 次のうち、更新系の実行メソッドとして正しいのはどれ？
 - A `executeQuery`
 - B `executeUpdate`
 - C `next`

▼ ヒント

(1) は「次の処理まで自動コミットが OFF のままだとどうなる？」を想像。(3) は「トランザクションの単位は何か？」を思い出す。

▼ 解答例

- (1) 例：自動コミットが OFF のままだと、次に実行する更新が意図せず同じトランザクションに入ったり、`commit` し忘れて反映されないなど事故が起きるため。
- (2) `commit`：全部の更新が成功したとき。`rollback`：途中で失敗（`SQLException` など）したとき。
- (3) トランザクションは「同じ Connection で行った操作」をまとめる仕組みで、Connection が違うと別トランザクションになるから。
- (4) B